

Deep Learning

Deep Sequential Models

Deep Sequential Models

Traditional feedforward neural networks have two important limitations:

- **Fixed-size inputs and outputs**: They typically require input tensors of a fixed size and produce outputs of a predetermined size.
- **Inappropriate inductive bias for sequential data:** Their architecture does not explicitly encode the importance of **temporal order or dependencies between successive elements** (as if the features were interchangeable).

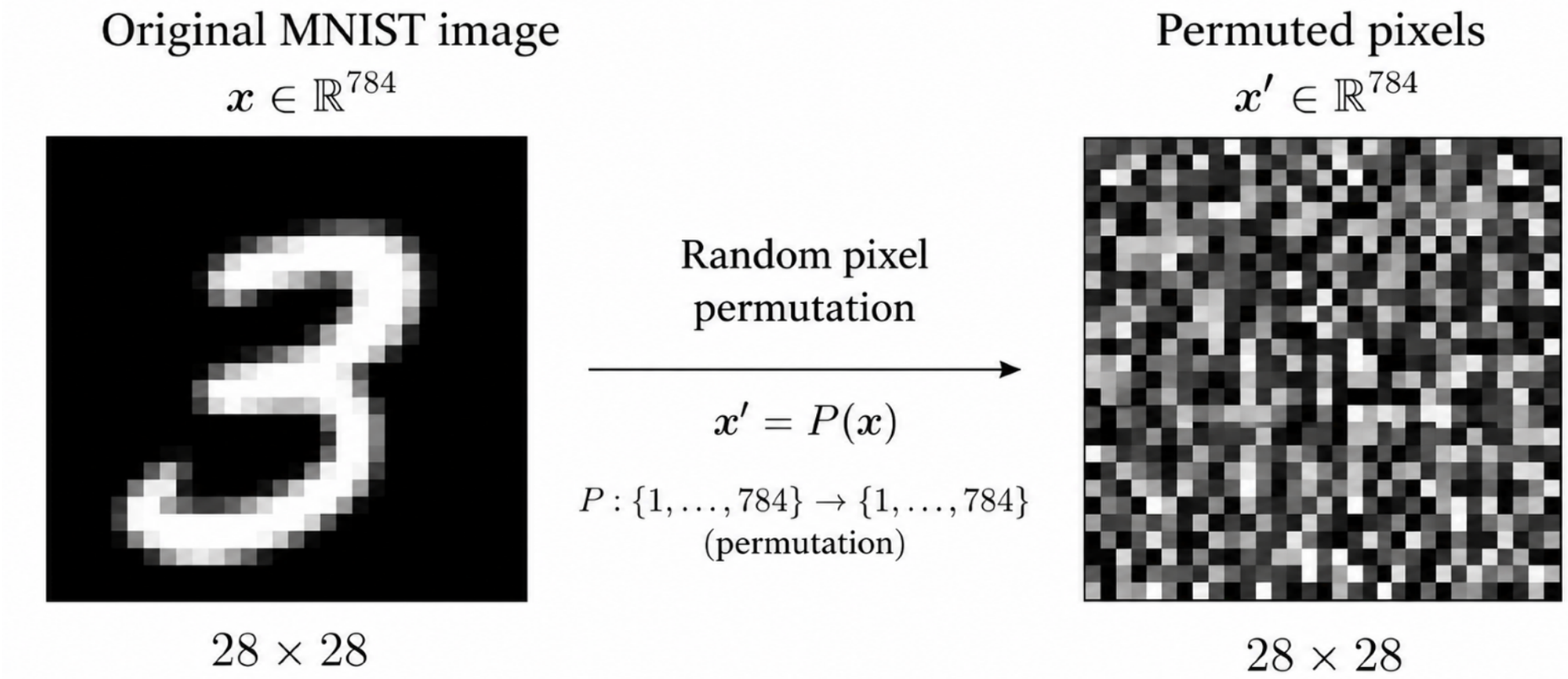
Deep Sequential Models

Consider a 28 × 28 MNIST image, represented as a vector of 784 pixel intensities: $\mathbf{x} = (x_1, x_2, \ldots, x_{784})$.

A traditional fully connected neural network treats each pixel as a separate input feature.

- **Fixed input size:** The network expects exactly 784 input features and cannot directly process images of different dimensions.
- **No spatial inductive bias**: The network does not explicitly know that x_{15} and x_{16}correspond to neighboring pixels, or that nearby pixels are more likely to be related than distant ones. We could apply a fixed random pixel permutation to all MNIST images and train a new fully connected network with essentially the same expressive power. The architecture itself does not privilege the original spatial arrangement.

Deep Sequential Models



A fully connected neural network can learn correlations between any pair or group of pixels, including neighboring ones. However, it has **no built-in preference for relationships between nearby pixels over those between distant pixels**.

Consequently, applying a fixed permutation to the input pixels does not change the network's expressive power. **Convolutional neural networks (CNNs) explicitly encode this spatial preference as an inductive bias**.

Deep Sequential Models

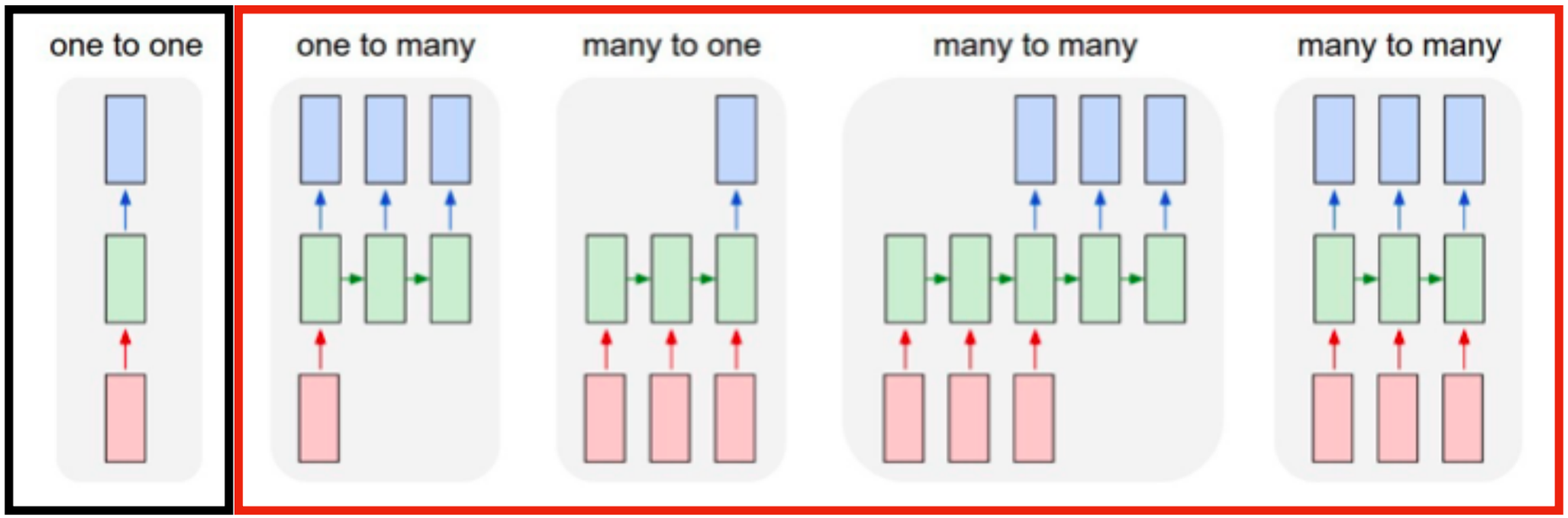
Recurrent neural networks (RNNs) address these limitations by processing **sequences** of tensors, maintaining a **hidden state** that captures information from previous steps.

This allows them to handle **variable-length sequences and model temporal dependencies**.

Deep Sequential Models

FCN/CNN ARCHITECTURE

Deep Sequential ARCHITECTURES



1 fixed-size tensor

→

1 fixed-size tensor

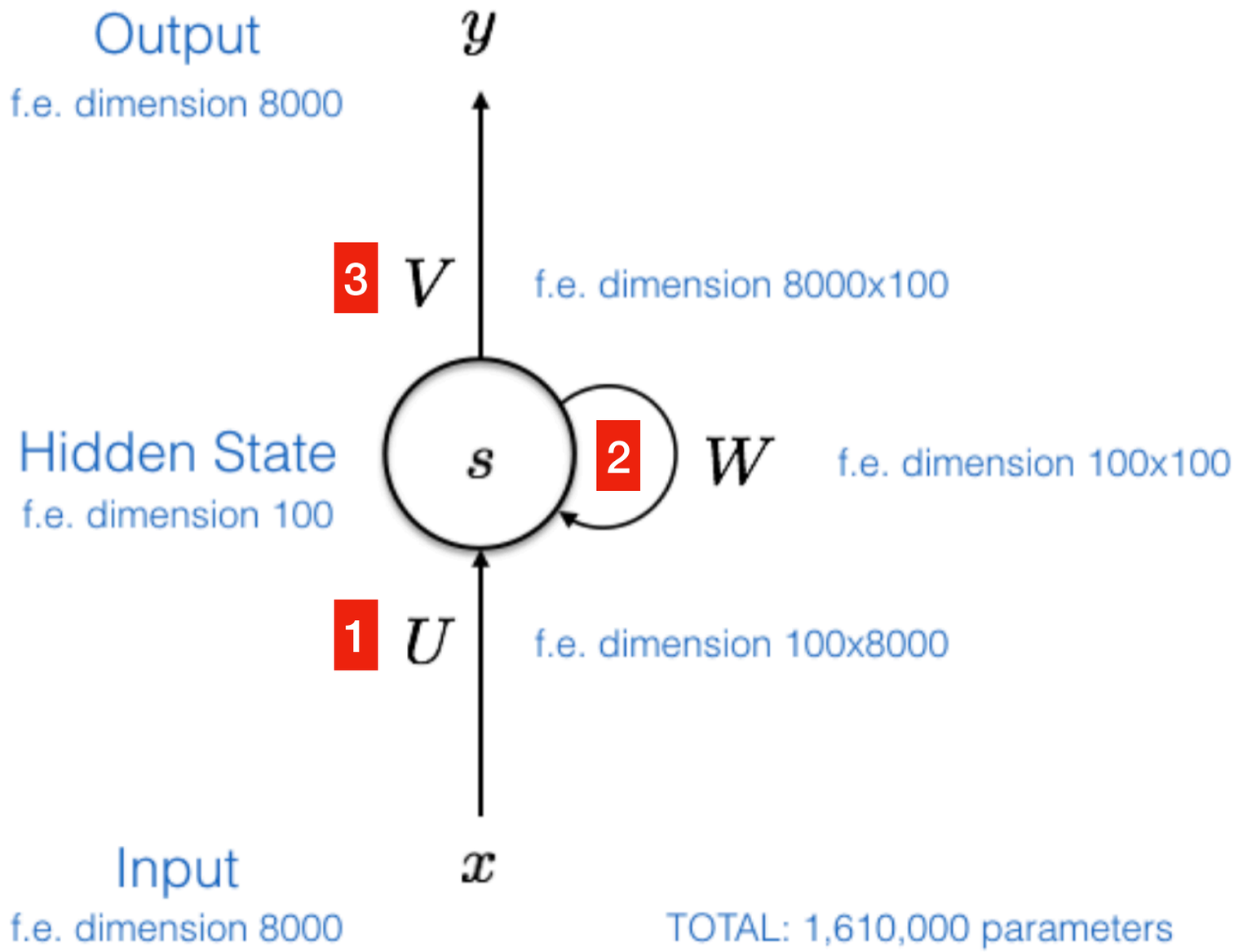
variable-size tensors

→

variable-size tensors

First approach: Vanilla RNN

Example: Recurrent model that operates on sequences of words (vocabulary = 8000 different words)

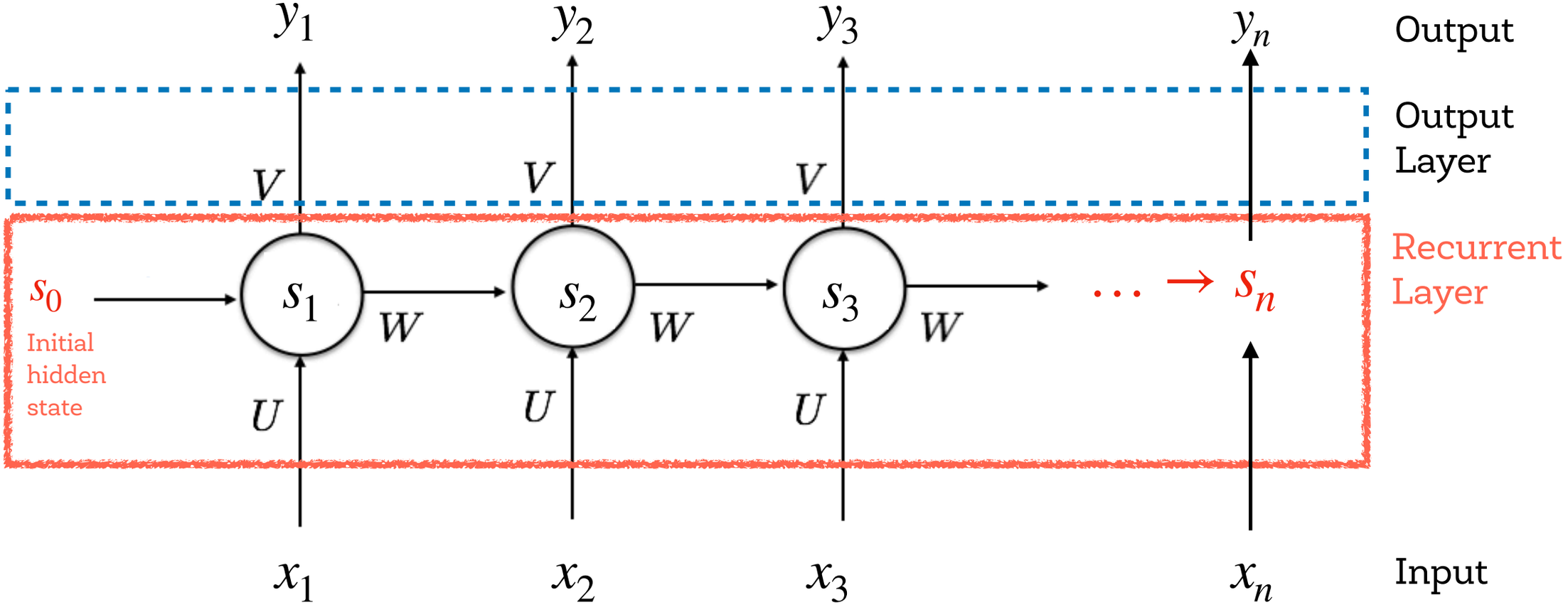


$$\mathbf{x} = (x_1, \ldots, x_n)$$

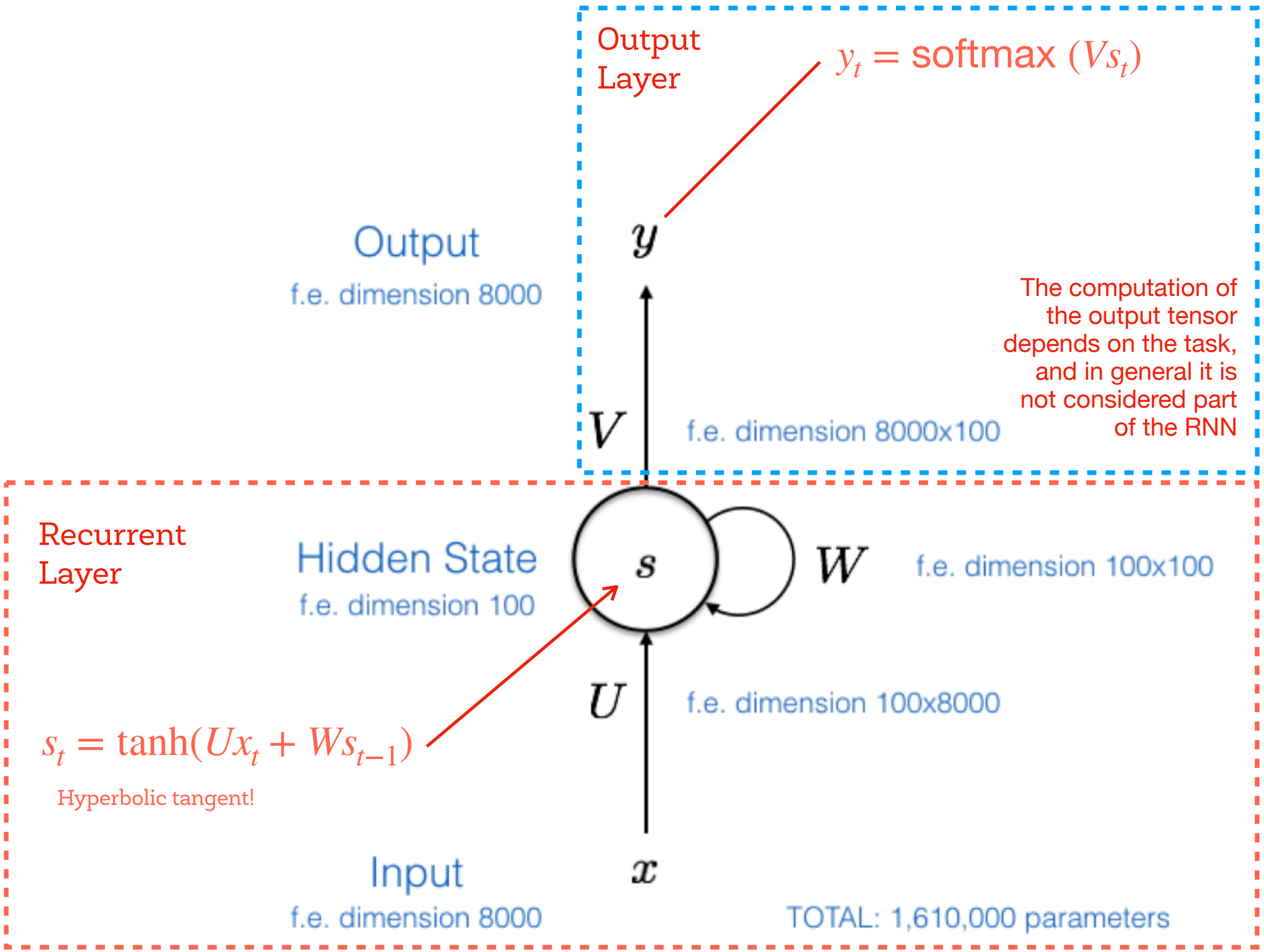
Vanilla RNN

Instead of imagining that hidden state is being recurrently fed back into the network, it's easier to visualize the process if we **unroll** the operation into a computational graph that is composed to many time steps.

By unrolling we mean that we write out the network for the complete sequence.



Vanilla RNN


Output
Layer
$y_t = \text{softmax}\,(Vs_t)$
Output
f.e. dimension 8000
y
The computation of the output tensor depends on the task, and in general it is not considered part of the RNN
V
f.e. dimension 8000x100
Recurrent
Layer
Hidden State
f.e. dimension 100
s
W
f.e. dimension 100x100
U
f.e. dimension 100x8000
$s_t = \tanh(Ux_t + Ws_{t-1})$
Hyperbolic tangent!
Input
f.e. dimension 8000
x
TOTAL: 1,610,000 parameters

Vanilla RNN

- We can think of the **hidden state** s_t as a memory of the network that **captures/stores information about the previous steps**.

- The RNN **shares the parameters** U, V, W across all time steps.

- It is not necessary to have outputs y_t at each time step.

- The value of s_0 is a learned parameter of the model.

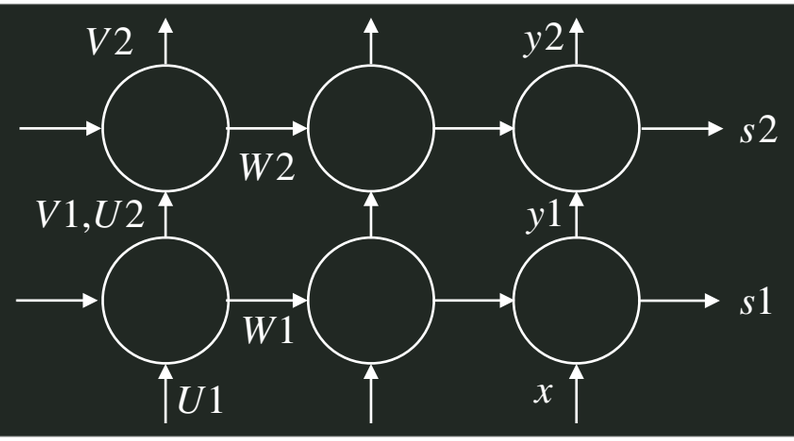
Vanilla RNN

```
def RNN_step(x):
    s = np.tanh(np.dot(W, s) + np.dot(U, x))
    y = np.dot(V, h)
    return y
```

This is heavy model because of the matrices!

We can go deep by stacking RNNs:

```
y1 = RNN.step(x)
y2 = RNN.step(y1)
```



Vanilla RNN

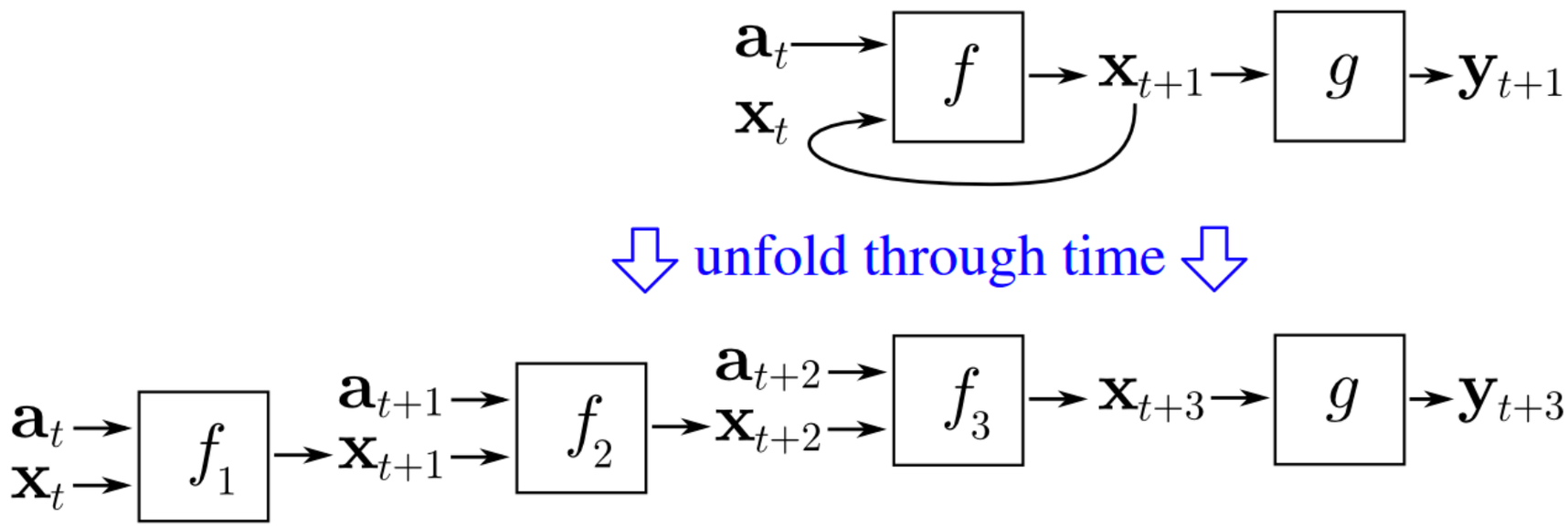
NumPy implementation of a simple RNN

```
import numpy as np
timesteps = 100
input_features = 32
output_features = 64
inputs = np.random.random((timesteps, input_features))
state_t = np.zeros((output_features,))
W = np.random.random((output_features, input_features))
U = np.random.random((output_features, output_features))
b = np.random.random((output_features,))
successive_outputs = []
for input_t in inputs:
    output_t = np.tanh(np.dot(W, input_t) + np.dot(U, state_t) + b)
    successive_outputs.append(output_t)
    state_t = output_t
final_output_sequence = np.concatenate(successive_outputs, axis=0)
```

Vanilla RNN

Training a RNN is similar to training a traditional NN, but with some modifications.

The main reason is that **parameters are shared by all time steps**: in order to compute the gradient at $t = 4$, we need to propagate 3 steps and **sum up the gradients**. This is called **Backpropagation through time (BPTT)**.



https://en.wikipedia.org/wiki/Backpropagation_through_time#/media/File:Unfold_through_time.png

Vanilla RNN for language modeling

Let's suppose we are processing a **series of words**:

$$x_1, \ldots, x_{t-1}, x_t, x_{t+1}, \ldots x_T$$

x_i are the one-hot word **vectors** corresponding to a corpus with d **symbols**.

Then, the relationship to compute the hidden layer output features at each time-step t is $h_t = \sigma(W^{(hh)}h_{t-1} + W^{(hx)}x_t)$, where:

- $x_t \in \mathbb{R}^d$ is input word vector at time t.
- $W^{(hx)} \in \mathbb{R}^{D_h \times d}$ is the weights matrix used to condition the input word vector, x_t.
- $W^{(hh)} \in \mathbb{R}^{D_h \times D_h}$ is the weights matrix used to condition the output of the previous time-step, h_{t-1}.
- $h_{t-1} \in \mathbb{R}^{D_h}$ is the output of the non-linear function at the previous time-step, $t-1$.
- $h_0 \in \mathbb{R}^{D_h}$ is an initialization vector for the hidden layer at time-step $t = 0$.
- $\sigma()$ is the non-linearity function (normally, *tanh*).

Vanilla RNN for language modeling

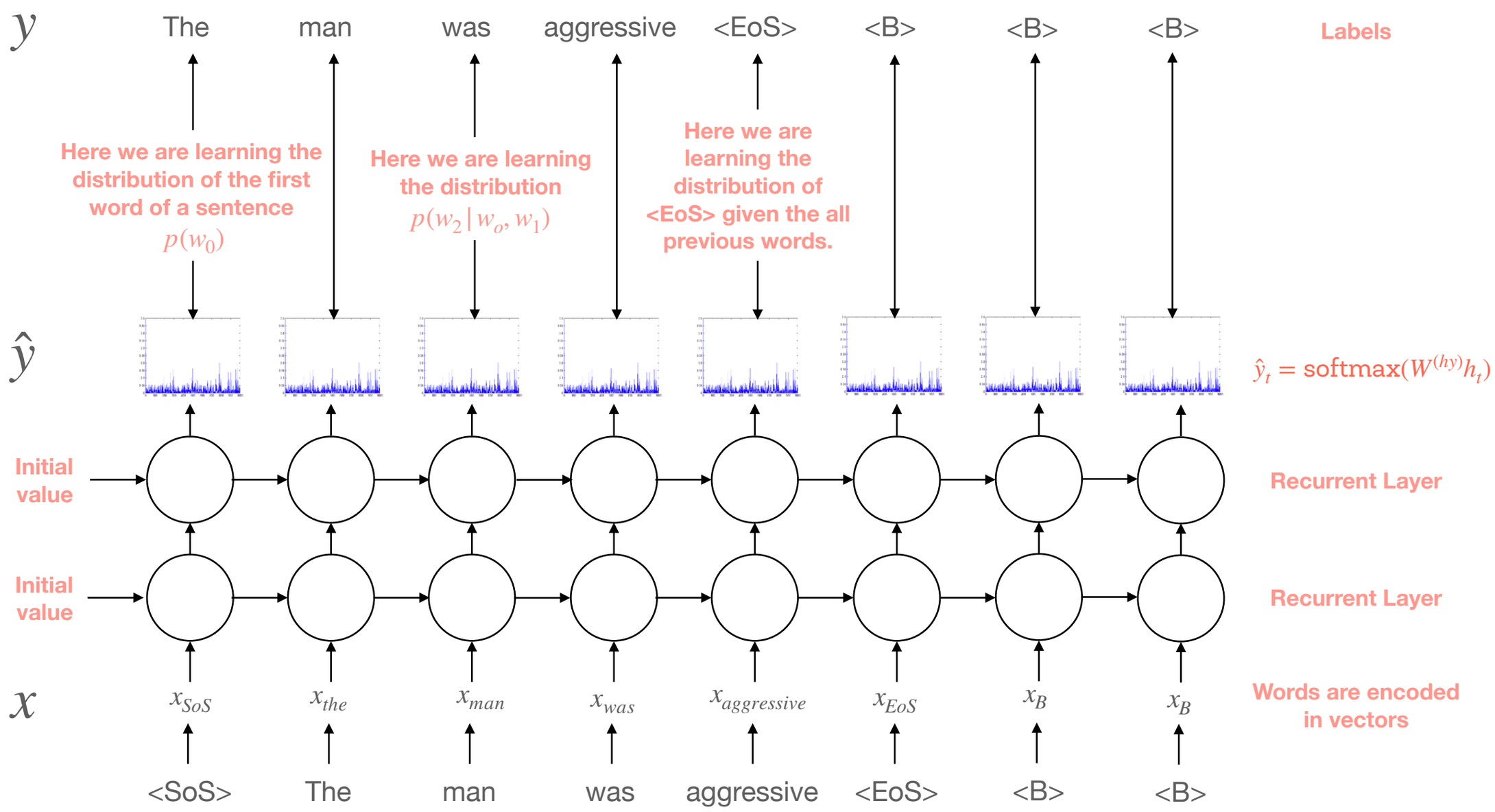
We can compute the **output probability distribution over the vocabulary** at each time-step t as $\hat{y}_t = softmax(W^{(hy)}h_t)$.

Essentially, $\hat{y}_t$ is the next predicted word given the sequence context score so far (i.e. h_{t-1}) and the last observed word vector $x^{(t)}$. Here, $W^{(hy)} \in \mathbb{R}^{d \times D_h}$ and $\hat{y} \in \mathbb{R}^d$.

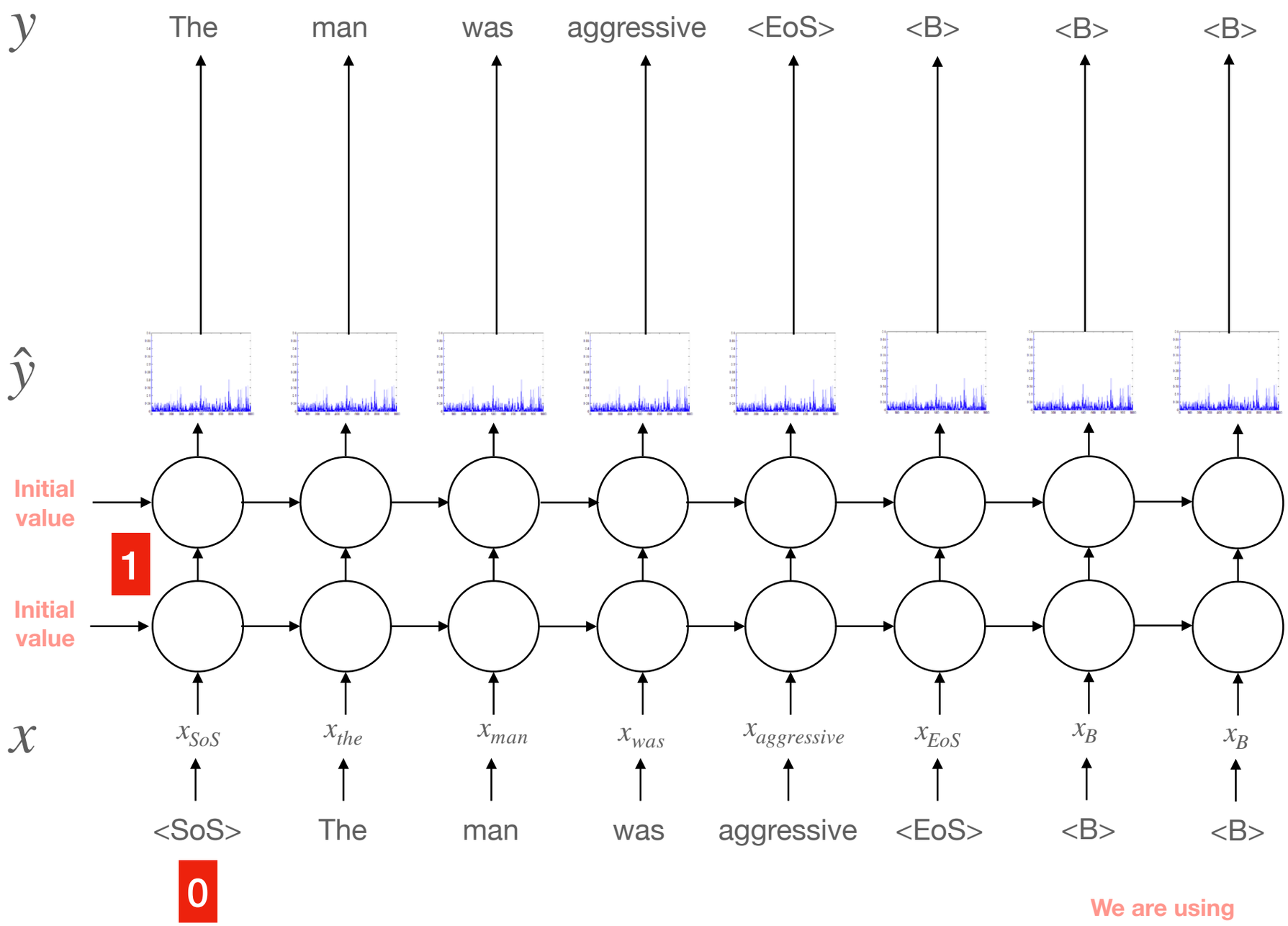
Vanilla RNN for language modeling

This is a RNN that implements a **language model** (a probability distribution over word sequences) that can be sampled.

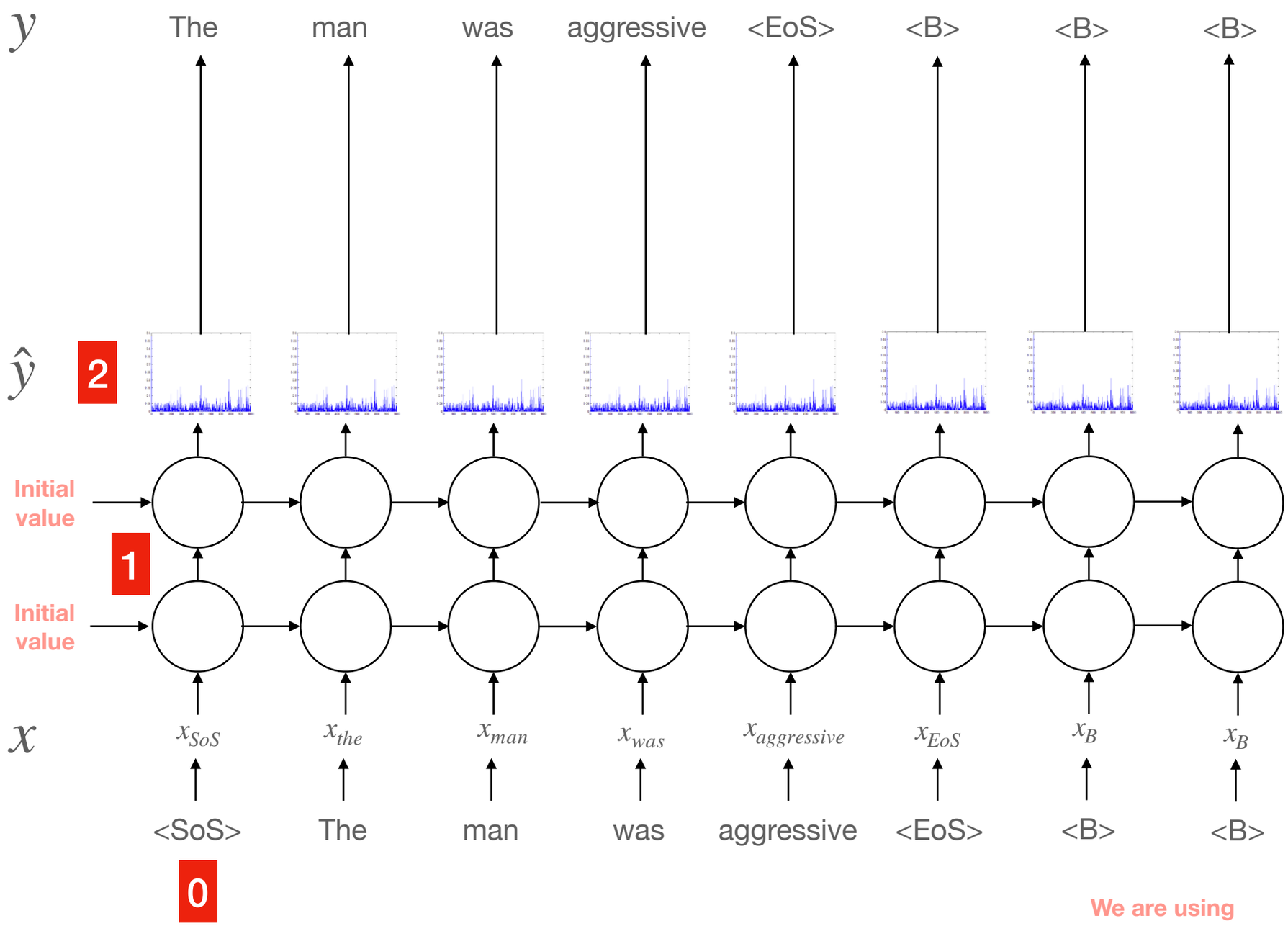
$$P(w_0, w_1, \ldots, w_n) = P(w_0)P(w_1 \mid w_0)P(w_2 \mid w_0, w_1), \ldots P(w_n \mid w_0, \ldots, w_{n-1})$$



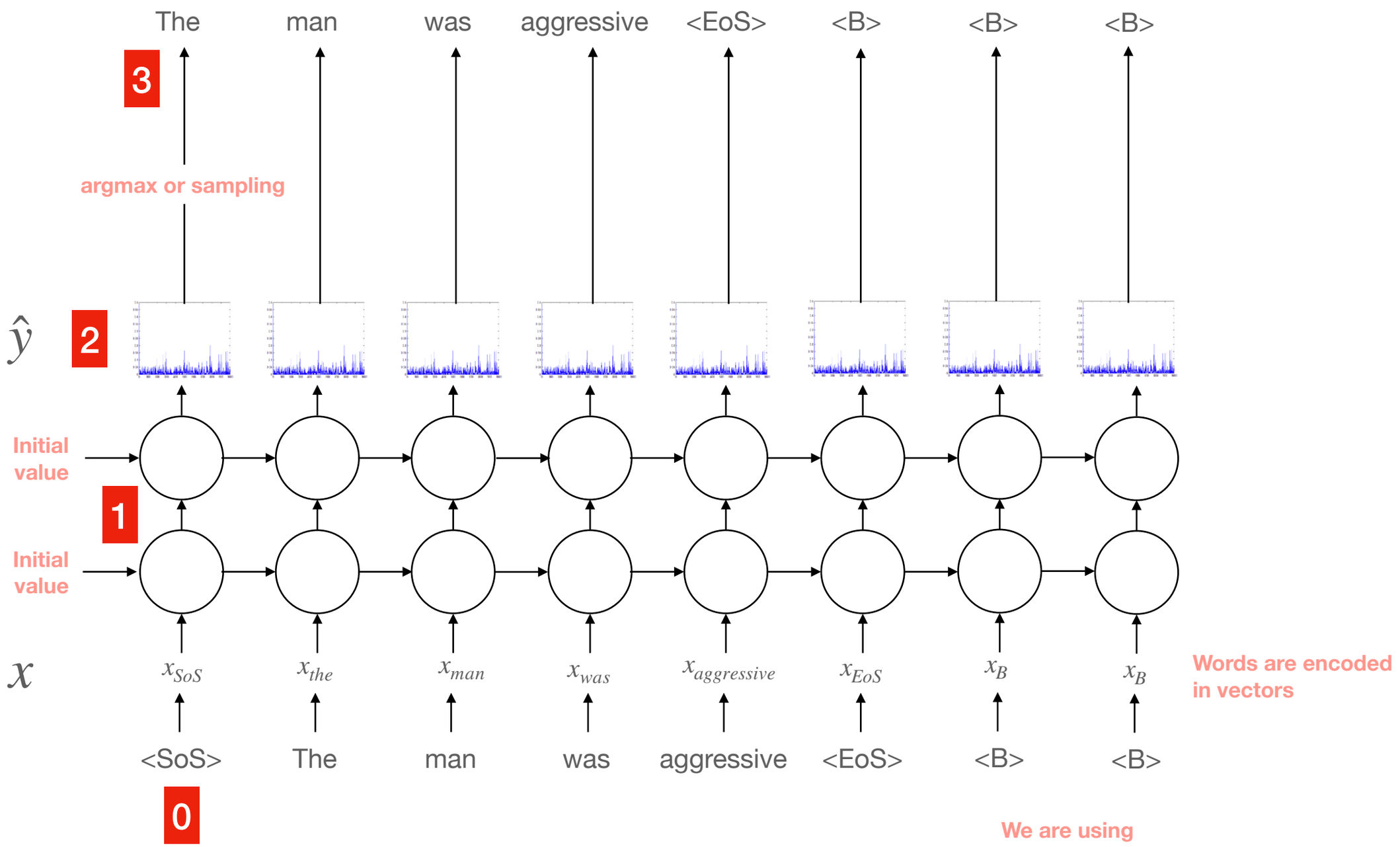
Vanilla RNN for language modeling



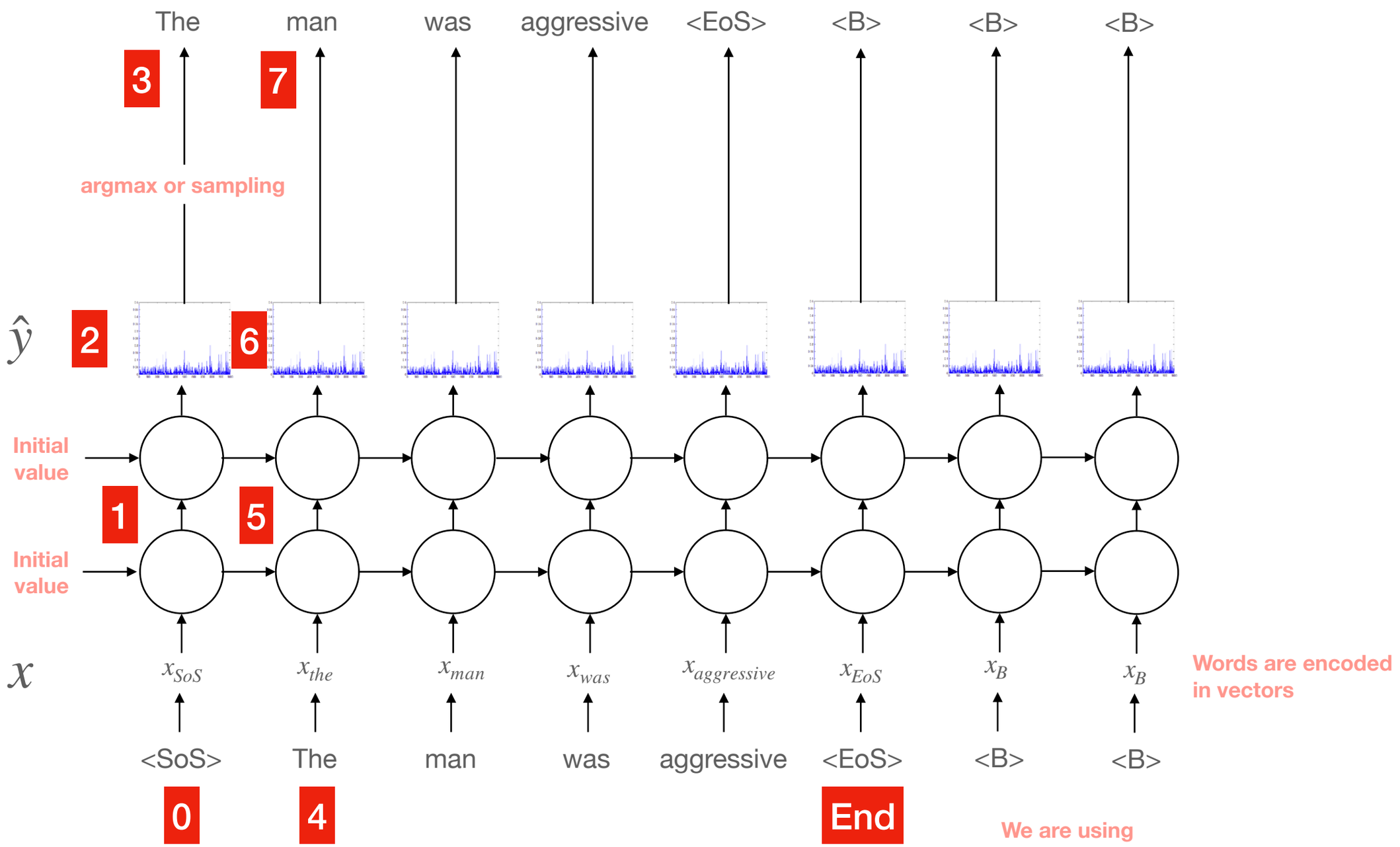
Vanilla RNN for language modeling



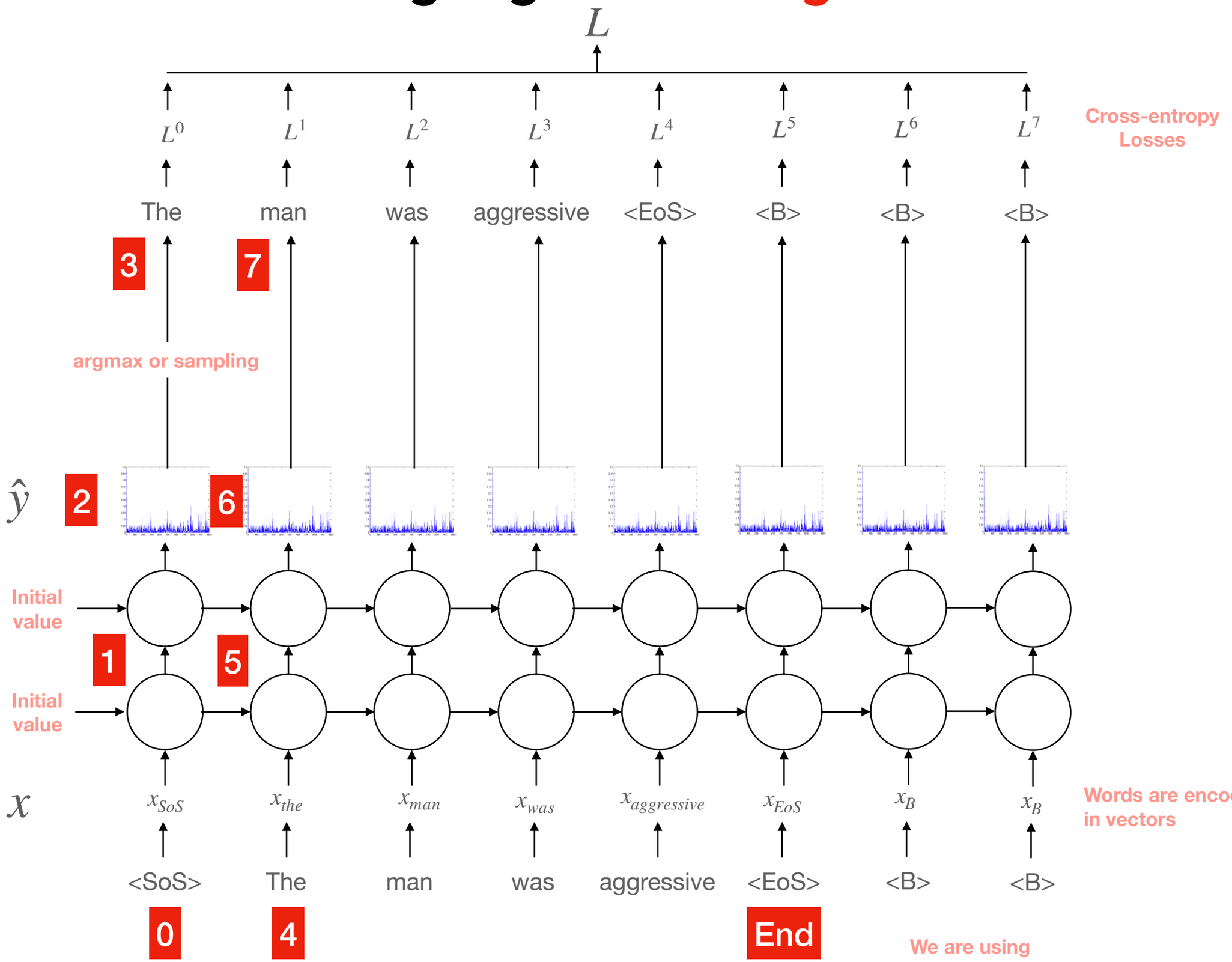
Vanilla RNN for language modeling



Vanilla RNN for language modeling



Vanilla RNN for language modeling



Vanilla RNN for language modeling

The loss function is the cross-entropy (classification) error:

$$L^{(t)}(W) = -\sum_{j=1}^{|V|} y_{t,j} \times log(\hat{y}_{t,j})$$

The cross entropy error over a sentence of size T is:

$$L = \frac{1}{n}\sum_{t=1}^{n} L^{(t)}(W) = -\frac{1}{n}\sum_{t=1}^{n}\sum_{j=1}^{|V|} y_{t,j} \times log(\hat{y}_{t,j})$$

Softmax

Given a numerical output $\mathbf{z} = (z_1, \ldots, z_K)$ representing a vector of classification scores for K classes, the softmax function can be used to convert these scores into probabilities:

$$p_k = \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

Question: Why do we use softmax instead of simple normalization?

$$\text{Norm}(\mathbf{z}) = \frac{\mathbf{z}}{\sum_{j=0}^{K-1} z_j}$$

Answer: Because softmax has better properties for stochastic gradient descent than simple normalization.

Cross entropy loss

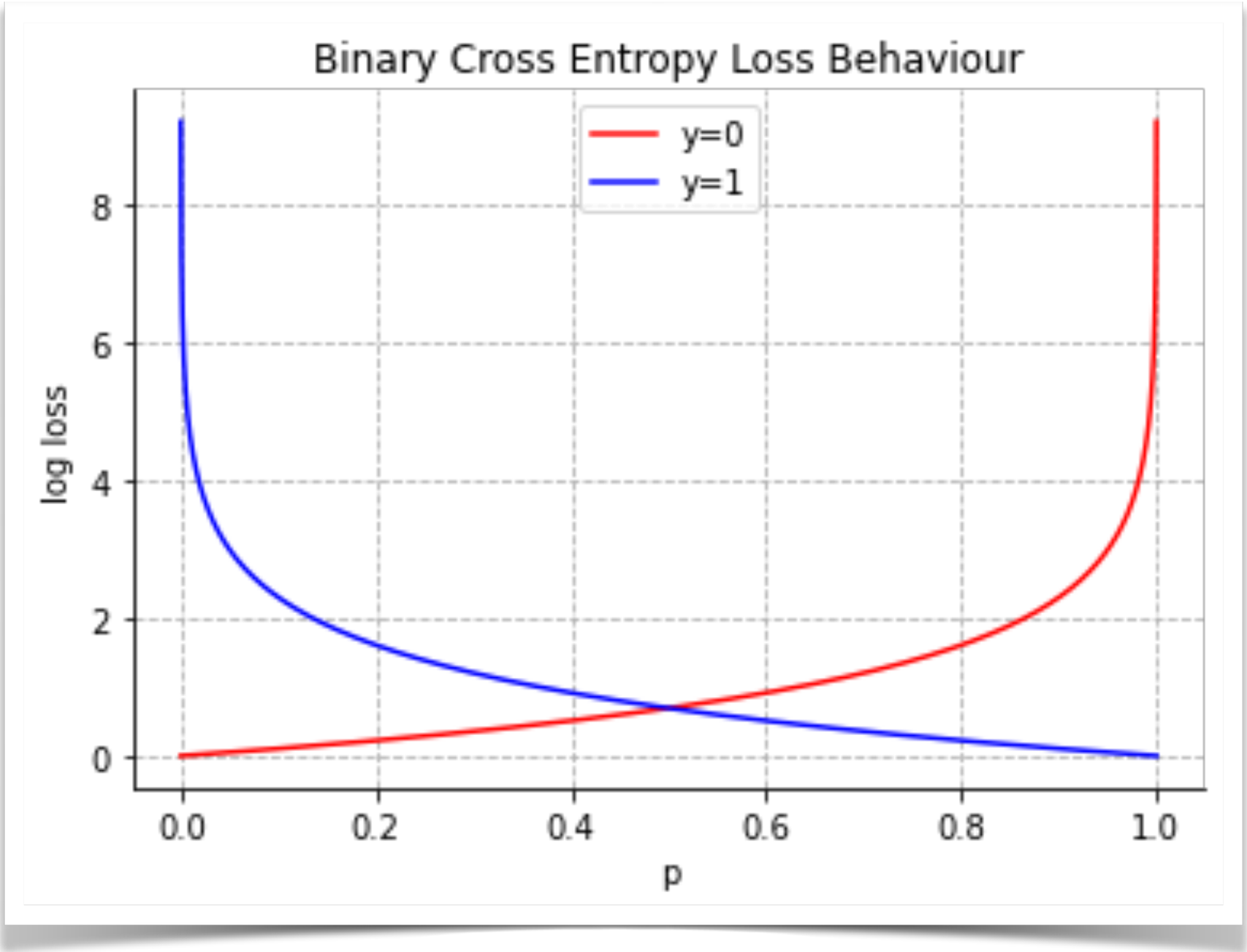
Given a dataset $(\mathbf{x}_i, y_i)$ with n samples, features $\mathbf{x} \in \mathbb{R}^m$ and labels $y \in \{0,1\}$, the model outputs a value between 0 and 1.

The **binary cross-entropy loss** function is defined as:

$$CE = -\frac{1}{n}\sum_{i=1}^{n}\left(y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right)$$

where $\hat{y}$ is the predicted probability of class 1, and $1-\hat{y}$ is the predicted probability of class 0.

Cross entropy loss



$$CE = -\frac{1}{n}\sum_{i=1}^{n}\left(y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right)$$

Cross entropy loss

This loss function can be generalized to the multiclass case.

Given a dataset $(\mathbf{x}_i, y_i)$ with n samples, features $\mathbf{x} \in \mathbb{R}^m$ and labels $y \in \{1,2,\ldots,K\}$, the cross-entropy loss function is defined as:

$$CE = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{K} y_i^j \log(\hat{y}_i^j)$$

Here, y_i^j is either 0 or 1, indicating whether j is the correct class for sample $\mathbf{x}_i$ (one-hot model), while $\hat{y}_i^j$ is the predicted probability of class j for $\mathbf{x}_i$. Binary cross-entropy corresponds to the special case where K=2.

Cross entropy loss

For K classes, the model outputs a probability distribution:

$$q = (q_1, \ldots, q_K).$$

The correct class is typically represented using a one-hot encoded label p. Therefore, the cross-entropy loss is computed as:

$$CE = -\sum_{k=1}^{K} p_k \log q_k = -\log q_y$$

The higher the probability assigned to the correct class, the lower the loss.

Vanilla RNN

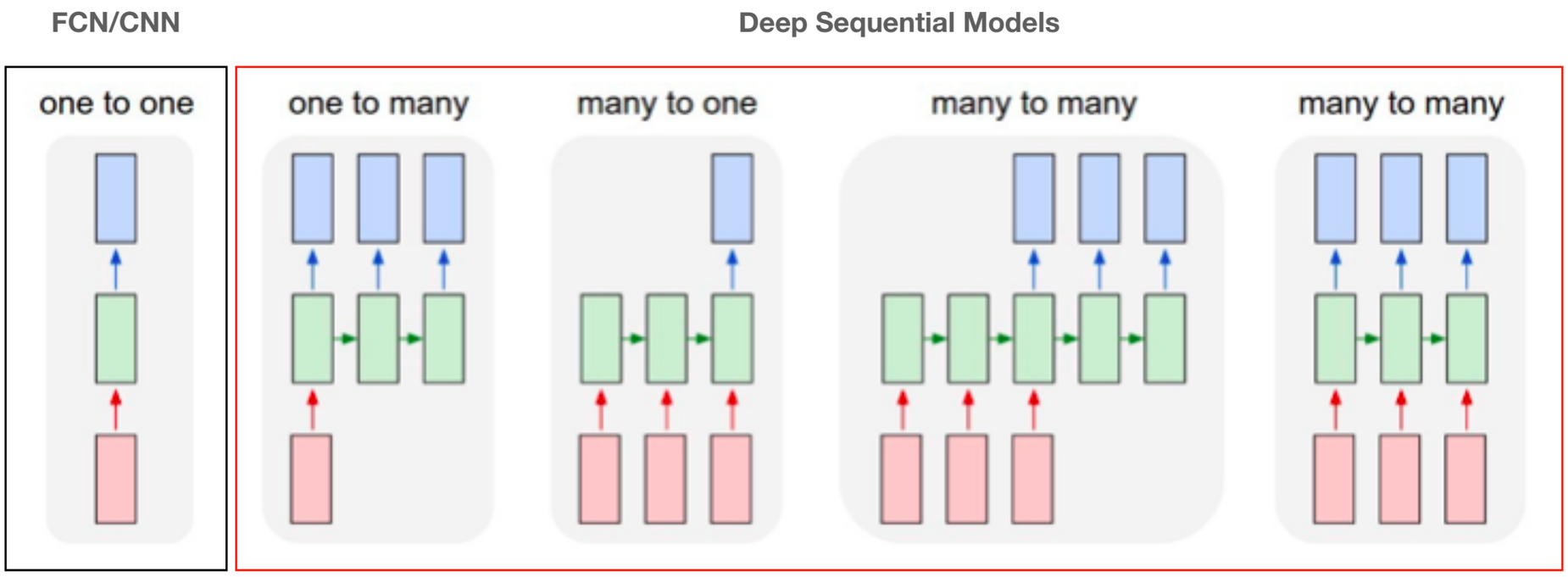
```
tf.keras.layers.SimpleRNN(
    units,
    activation="tanh",
    use_bias=True,
    kernel_initializer="glorot_uniform",
    recurrent_initializer="orthogonal",
    bias_initializer="zeros",
    kernel_regularizer=None,
    recurrent_regularizer=None,
    bias_regularizer=None,
    activity_regularizer=None,
    kernel_constraint=None,
    recurrent_constraint=None,
    bias_constraint=None,
    dropout=0.0,
    recurrent_dropout=0.0,
    return_sequences=False,
    return_state=False,
    go_backwards=False,
    stateful=False,
    unroll=False,
    **kwargs
)
```

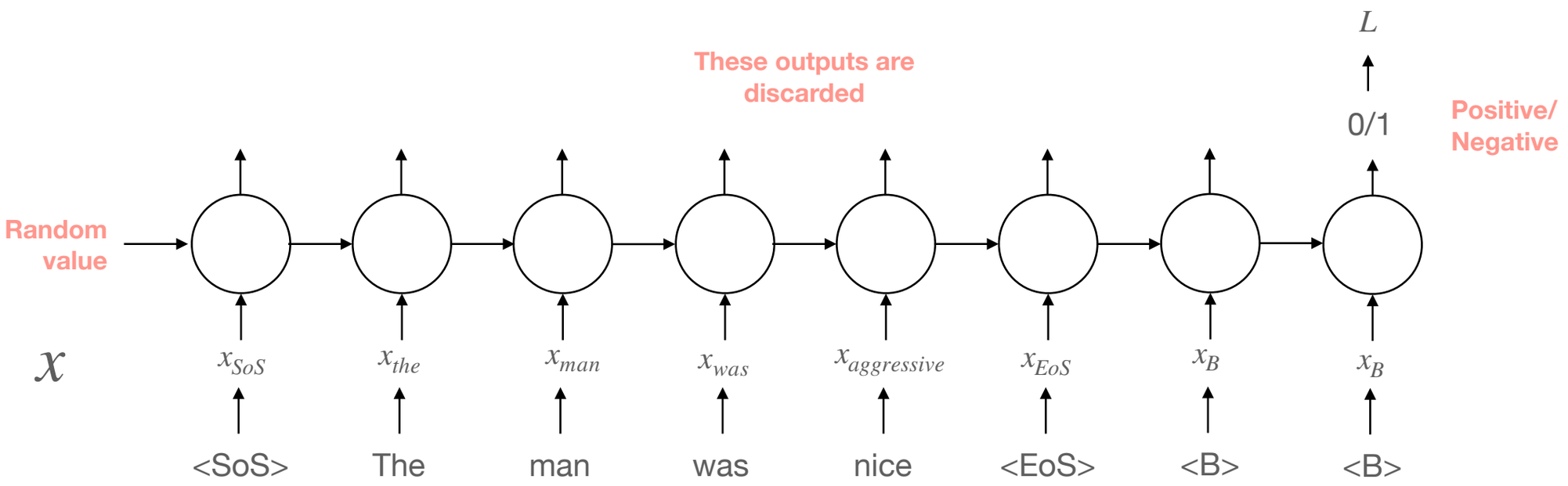
Vanilla RNN

```
tf.keras.layers.SimpleRNN(
    units,
    activation="tanh",
    use_bias=True,
    kernel_initializer="glorot_uniform",
    recurrent_initializer="orthogonal",
    bias_initializer="zeros",
    kernel_regularizer=None,
    recurrent_regularizer=None,
    bias_regularizer=None,
    activity_regularizer=None,
    kernel_constraint=None,
    recurrent_constraint=None,
    bias_constraint=None,
    dropout=0.0,
    recurrent_dropout=0.0,
    return_sequences=False,
    return_state=False,
    go_backwards=False,
    stateful=False,
    unroll=False,
    **kwargs
)
```

Draws samples from a uniform distribution within `[-limit, limit]`, where `limit = sqrt(6 / (fan_in + fan_out))` (`fan_in` is the number of input units in the weight tensor and `fan_out` is the number of output units).

Deep Sequential Models

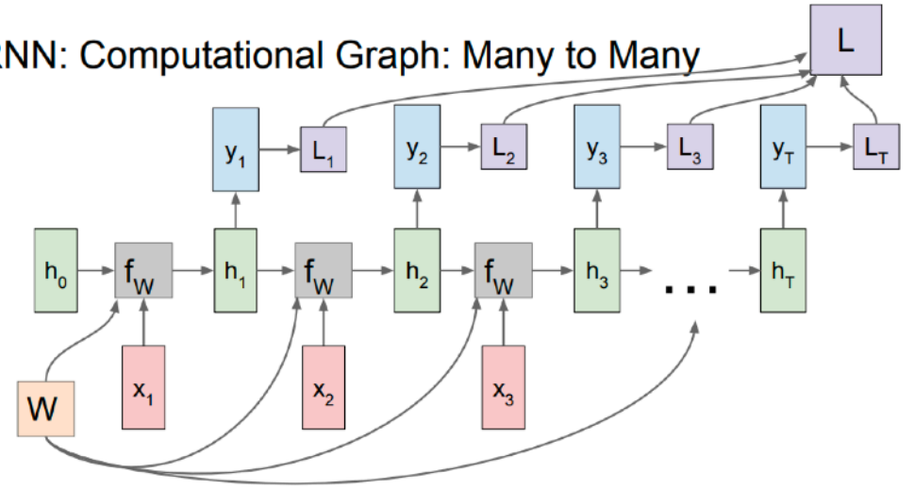




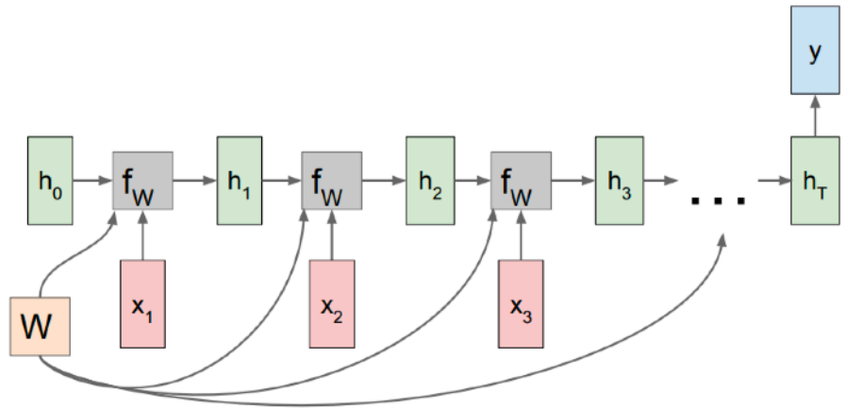
Example: Sentiment Analysis - Many to One

Deep Sequential Models

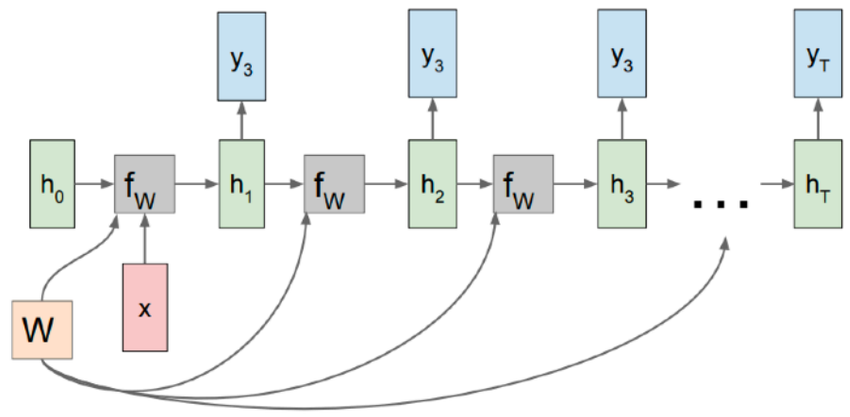
RNN: Computational Graph: Many to Many



RNN: Computational Graph: Many to One



RNN: Computational Graph: One to Many



https://calvinfeng.gitbook.io/machine-learning-notebook/supervised-learning/recurrent-neural-network/recurrent_neural_networks

RNN Training Tricks

Recurrent neural networks propagate weight matrices from one time-step to the next. Recall the goal of a RNN implementation is to enable propagating context information through faraway time-steps. When these propagation results in a **long series of matrix multiplications**, weights can vanish or explode.

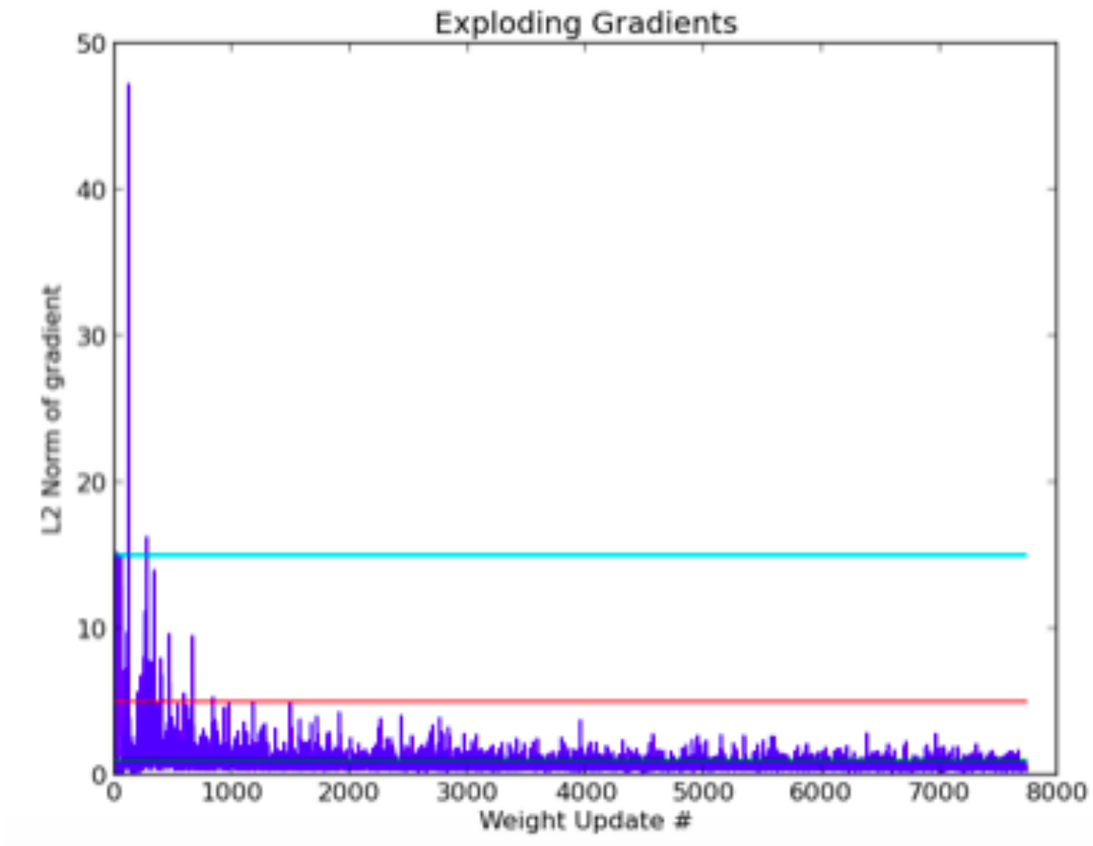
For example, when $||Wx||^2 \ll ||x||^2$ or $||Wx||^2 \gg ||x||^2$

Once the gradient value grows extremely large, it causes an overflow (i.e. NaN) which is easily detectable at runtime; this issue is called the **Gradient Explosion Problem**.

When the gradient value goes to zero, however, it can go undetected while drastically reducing the learning quality of the model for far-away words in the corpus; this issue is called the **Vanishing Gradient Problem**.

RNN Training Tricks

To solve the problem of **exploding gradients**, Thomas Mikolov first introduced a simple heuristic solution that clips gradients to a small number whenever they explode. That is, whenever they reach a certain threshold, they are set back to a small number.



RNN Training Tricks

To solve the problem of vanishing gradients, instead of initializing $W^{(hh)}$ randomly, starting off from **random orthogonal matrices** works better, i.e., a **random square matrix** W for which $W^T W = I$.

There are two properties of orthogonal matrices that are useful for training deep neural networks:

- they are norm-preserving, i.e., $\|Wx\|^2 = \|x\|^2$, and
- their columns (and rows) are all orthonormal to one another.

At least at the start of training, the first of these should help to keep the norm of the input constant throughout the network, which can help with the problem of exploding/vanishing gradients.

Similarly, an intuitive understanding of the second is that having orthonormal weight vectors encourages the weights to learn different input features.

RNN Training Tricks

You can obtain a random $n \times n$ orthogonal matrix W, (uniformly distributed) by performing a QR factorization of a $n \times n$ matrix with elements i.i.d. Gaussian random variables of mean 0 and variance 1.

```python
import numpy as np
from scipy.linalg import qr

n = 3
H = np.random.randn(n, n)
print(H)
print('\n')

Q, R = qr(H)

print (Q.dot(Q.T))
```

```
[[ 0.06351849  0.75175046 -0.10405964]
 [ 1.11485701  1.78923717 -0.9788983 ]
 [-0.08515308 -1.16475846  0.0640979 ]]


[[ 1.00000000e+00  5.49819565e-16 -1.29542567e-16]
 [ 5.49819565e-16  1.00000000e+00  6.99257794e-17]
 [-1.29542567e-16  6.99257794e-17  1.00000000e+00]]
```

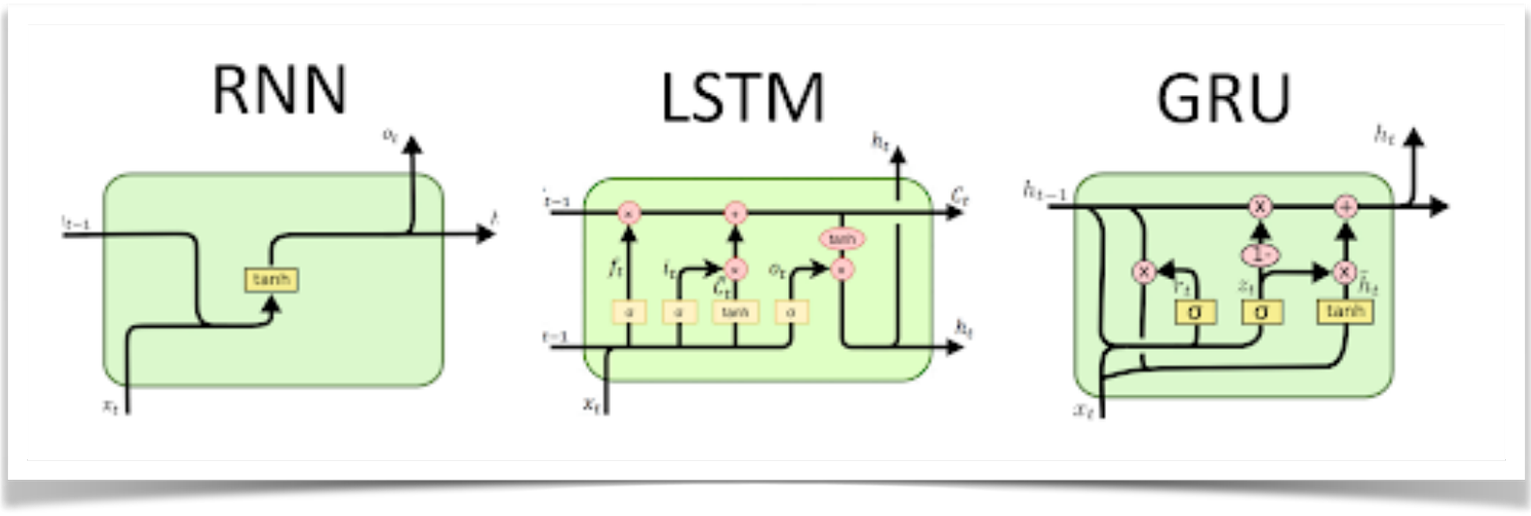
Vanilla RNN

- When you train the RNN for a task using gradient descent, the RNN model will not learn time dependencies longer than 2–3 steps in most cases.
- The reason are related to the fact that the activation update function for h_t is generally a contraction mapping.
- Consequently, whereas fixed-size neural networks (e.g. convnets) seem to have local optima that are almost as good as the global optima, RNNs have local optima that are extremely suboptimal.
- A number of improvements over vanilla RNNs, including the Gated Recurrent Unit (GRU) and Long Short-Term Memory (LSTM) networks, have helped solve these problems.

Gated Units

The most important types of **gated RNN**s are:

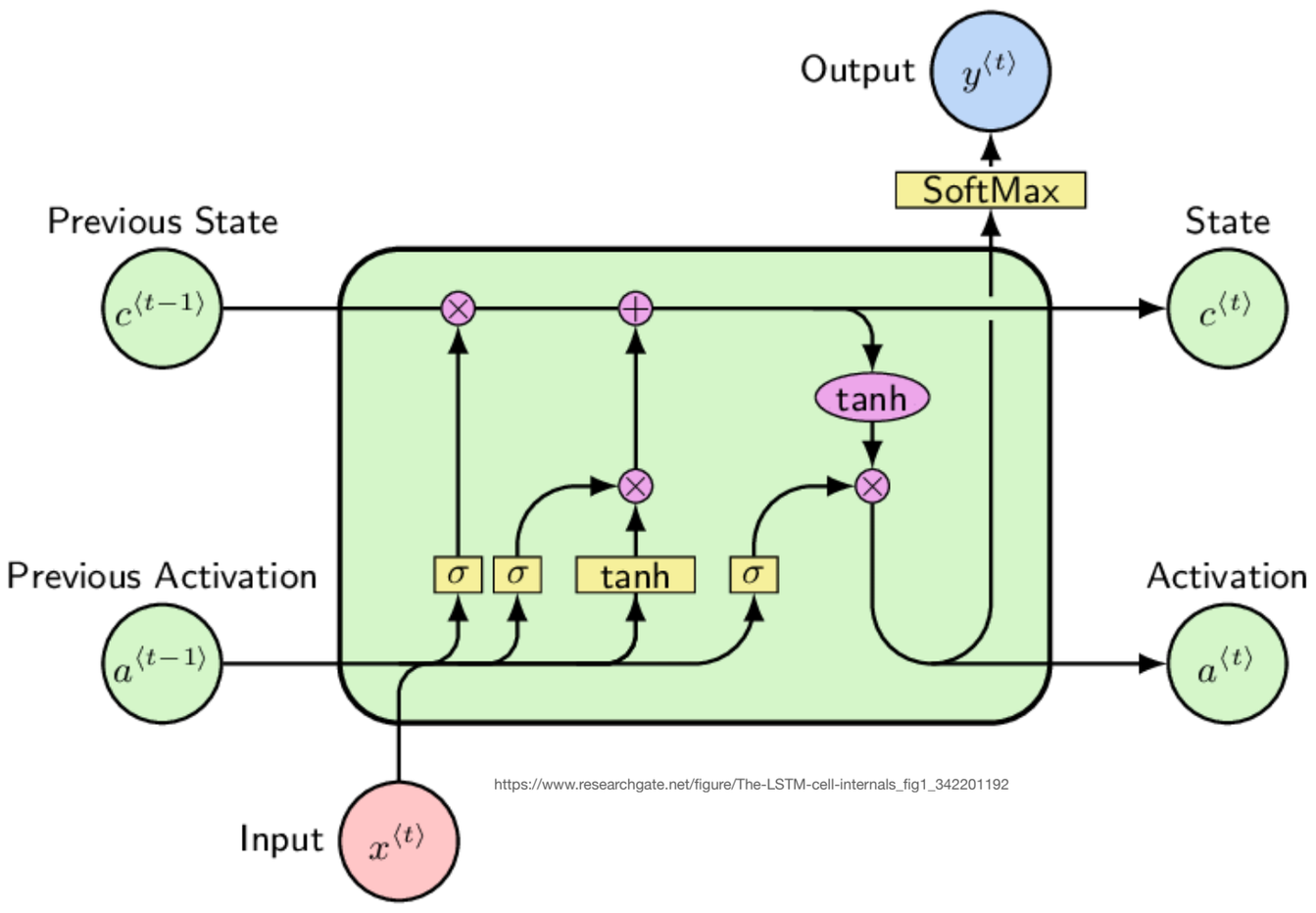
- **Long Short Term Memories** (LSTM). It was introduced by S.Hochreiter and J.Schmidhuber in 1997 and is widely used. LSTM is very good in the long run due to its high complexity.
- **Gated Recurrent Units** (GRU). It was introduced by K.Cho in 2014. It is simpler than LSTM, fasters and optimizes quicker.



LSTM

The key idea of LSTMs is the cell state C, the horizontal line running through the top of the diagram.

The cell state is kind of like a **conveyor belt**. It runs straight down the entire chain, with only some minor linear interactions. It's very easy for information to just flow along it unchanged.

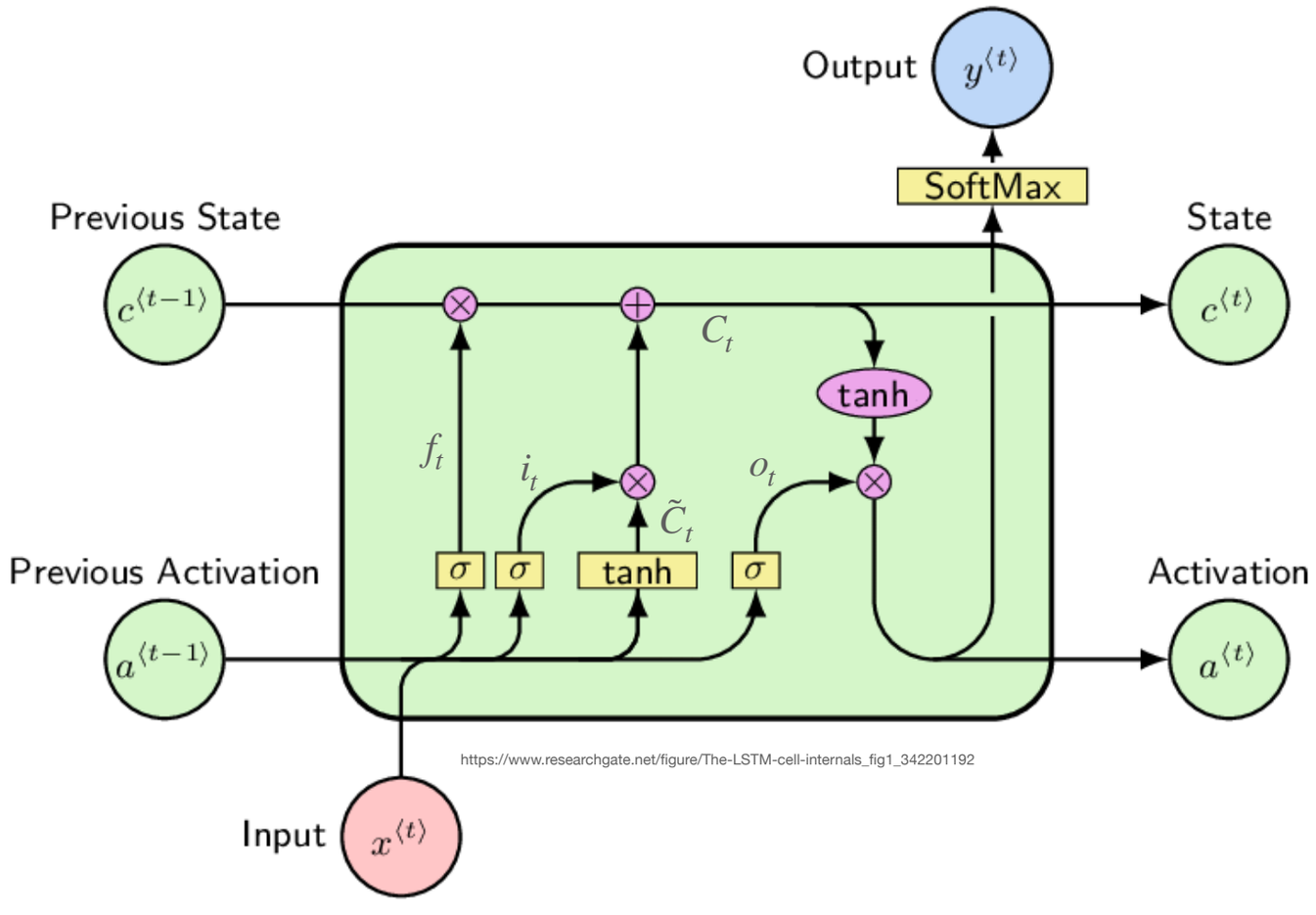


https://www.researchgate.net/figure/The-LSTM-cell-internals_fig1_342201192

LSTM

LSTM has the ability to **remove** or **add** information to the cell state, carefully regulated by structures called gates.

Gates are a way to optionally let information through. They are composed out of a **sigmoid** neural net layer and a pointwise multiplication operation.

- 

https://www.researchgate.net/figure/The-LSTM-cell-internals_fig1_342201192

$f_t = \sigma(W_f \cdot [a_{t-1}, x_t])$ (Forget gate)

$i_t = \sigma(W_i \cdot [a_{t-1}, x_t])$ (Input gate)

$\tilde{C}_t = \tanh(W_C \cdot [a_{t-1}, x_t])$

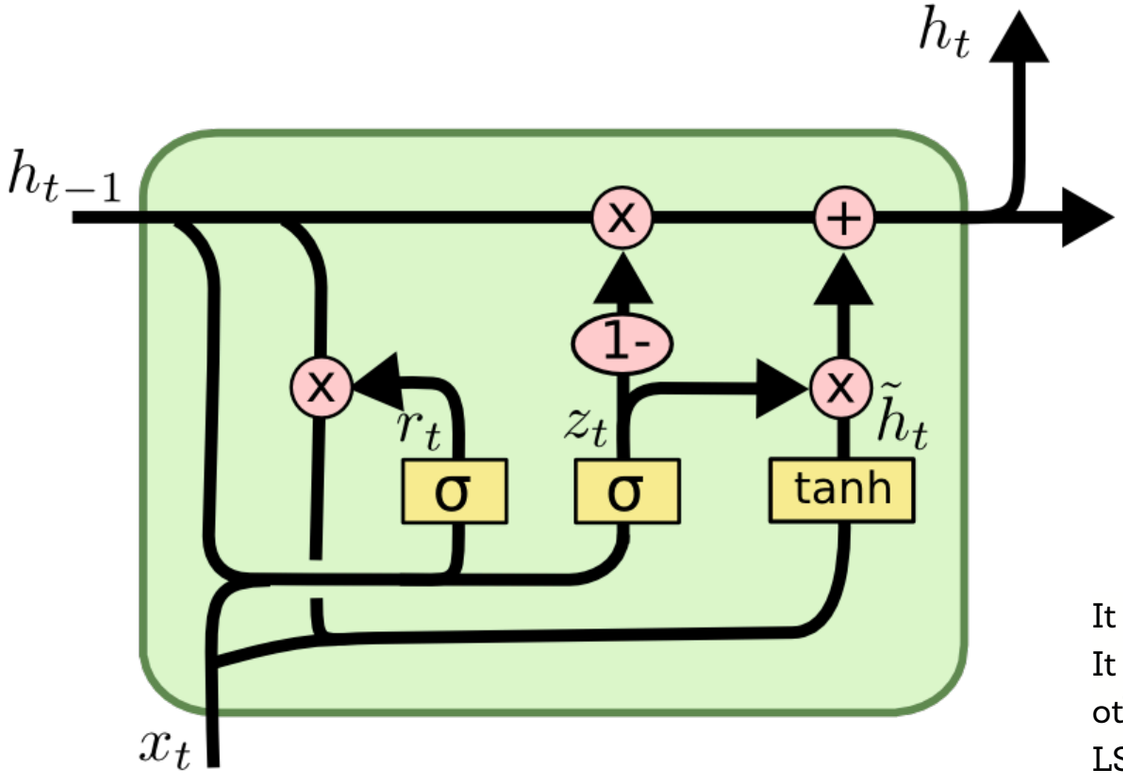
$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$ (Update gate)

$o_t = \sigma(W_o \cdot [a_{t-1}, x_t])$

$h_t = o_t * \tanh(C_t)$ (Output gate)

GRU

Gated recurrent units are designed in a manner to have persistent memory, like LSTMs, but they are lighter in terms of parameters.



$$z_t = \sigma(W_z \cdot [x_t, h_{t-1}]) \text{ (Update gate)}$$

$$r_t = \sigma(W_r \cdot [x_t, h_{t-1}]) \text{ (Reset gate)}$$

$$\tilde{h}_t = \tanh(r_t \cdot [x_t, r_t \circ h_{t-1}]) \text{ (New memory)}$$

$$h_t = (1 - z_t) \circ \tilde{h}_{t-1} + z_t \circ h_t \text{ (Hidden state)}$$

It combines the forget and input gates into a single “update gate.” It also merges the cell state and hidden state, and makes some other changes. The resulting model is simpler than standard LSTM models.

RNN in Keras

We can use LSTM/GRU layers with **multiple input sizes**. But, you need to process them before they are feed to the LSTM.

You need the **pad the sequences** of varying length to a fixed length. For this preprocessing, you need to determine the max length of sequences in your dataset.

You can do this in Keras with :

```
y = keras.preprocessing.sequence.pad_sequences(x,maxlen=10)
```

If the sequence is shorter than the max length, then zeros (default value) will appended till it has a length equal to the max length.

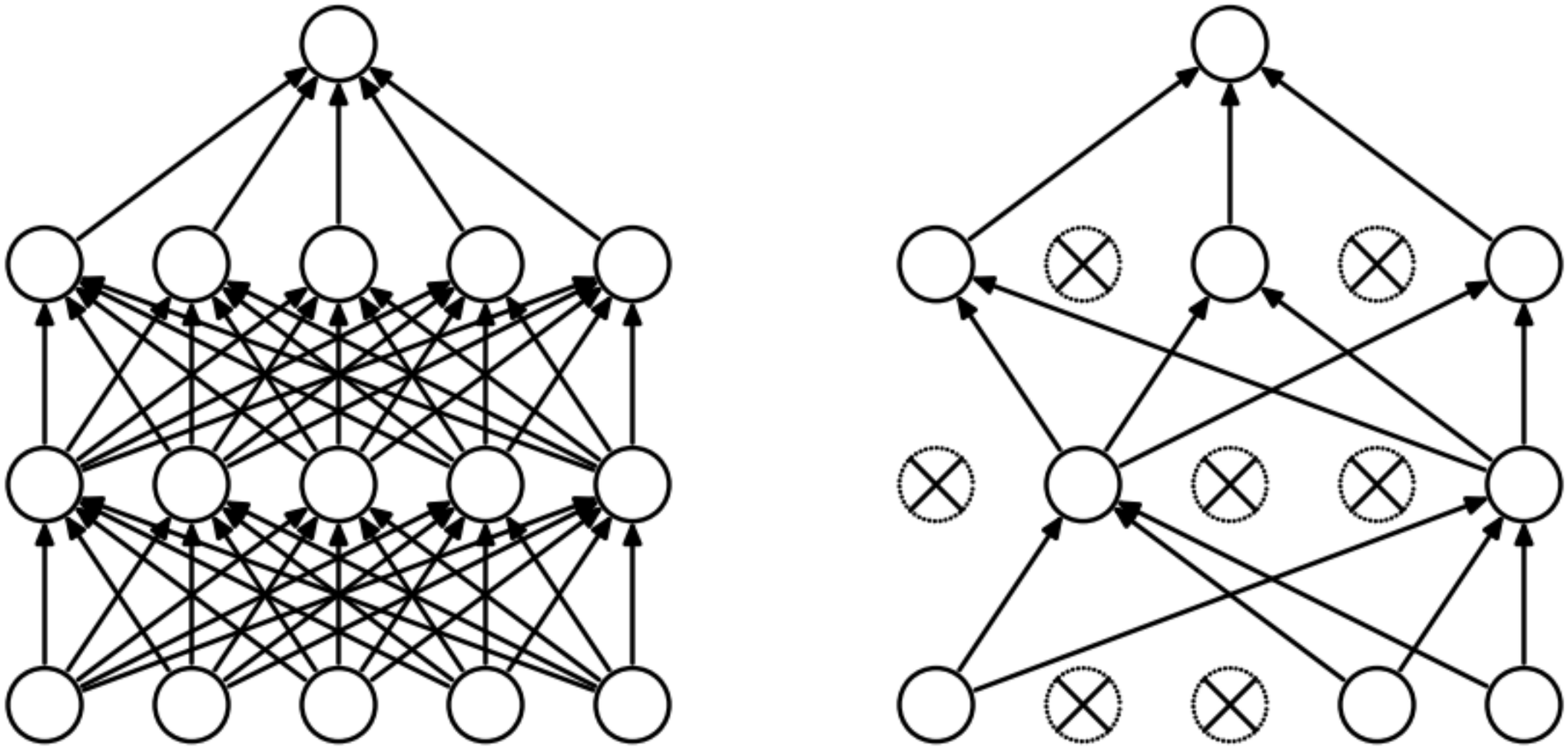
If the sequence is longer than the max length then, the sequence will be trimmed to the max length.

RNN in Keras

Advanced features:

- **Recurrent dropout** — A variant of dropout, used to fight overfitting in recurrent layers.
- **Stacking recurrent layers** — This increases the representational power of the model (at the cost of higher computational loads).
- **Bidirectional recurrent layers** — These present the same information to a recurrent network in different ways, increasing accuracy and mitigating forgetting issues.

RNN in Keras



(a) Standard Neural Net

(b) After applying dropout.

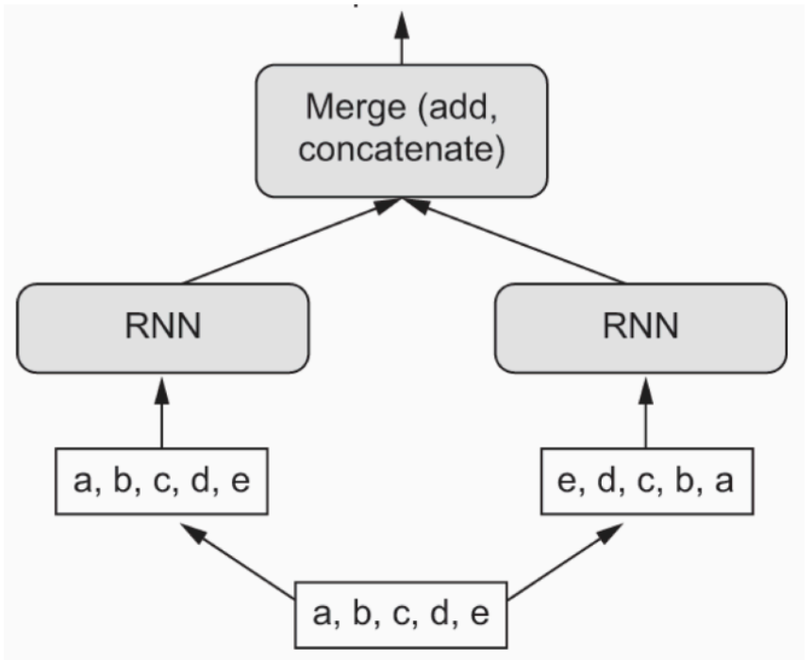
RNN in Keras

Bidirectional:

> RNNs are notably order-dependent: they process the time steps of their input sequences in order, and shuffling or reversing the time steps can completely change the representations the RNN extracts from the sequence.

A **bidirectional RNN** exploits the order sensitivity of RNNs: it consists of using two regular RNNs, such as the GRU and LSTM layers you're already familiar with, each of which processes the input sequence in one direction (chronologically and anti-chronologically), and then merging their representations.

RNN in Keras



The equations are (arrows are for designing left-to-right and right-to-left tensors):

$$\overrightarrow{h}_t = f(\overrightarrow{W}x_t + \overrightarrow{V}\overrightarrow{h}_{t-1} + \overrightarrow{b})$$
$$\hat{y}_t = g(Uh_t + c) = g(U[\overrightarrow{h}_t; \overleftarrow{h}_t] + c)$$

$[\overrightarrow{h}_t; \overleftarrow{h}_t]$ summarizes the past and future of a single element of the sequence.

Bidirectional RNNs can be stacked as usual!

Example in Keras

```
1 model = Sequential()
2
3 model.add(Embedding(vocab_size, embedding_dim))
4 model.add(Dropout(0.5))
5 model.add(LSTM(embedding_dim,return_sequences=True))
6 model.add(LSTM(embedding_dim))
7 model.add(Dense(6, activation='softmax'))
8
9 model.summary()
```

Turns nonnegative integers (indexes) into dense vectors of fixed size. eg [[4], [20]] -> [[0.25, 0.1], [0.6, -0.2]]

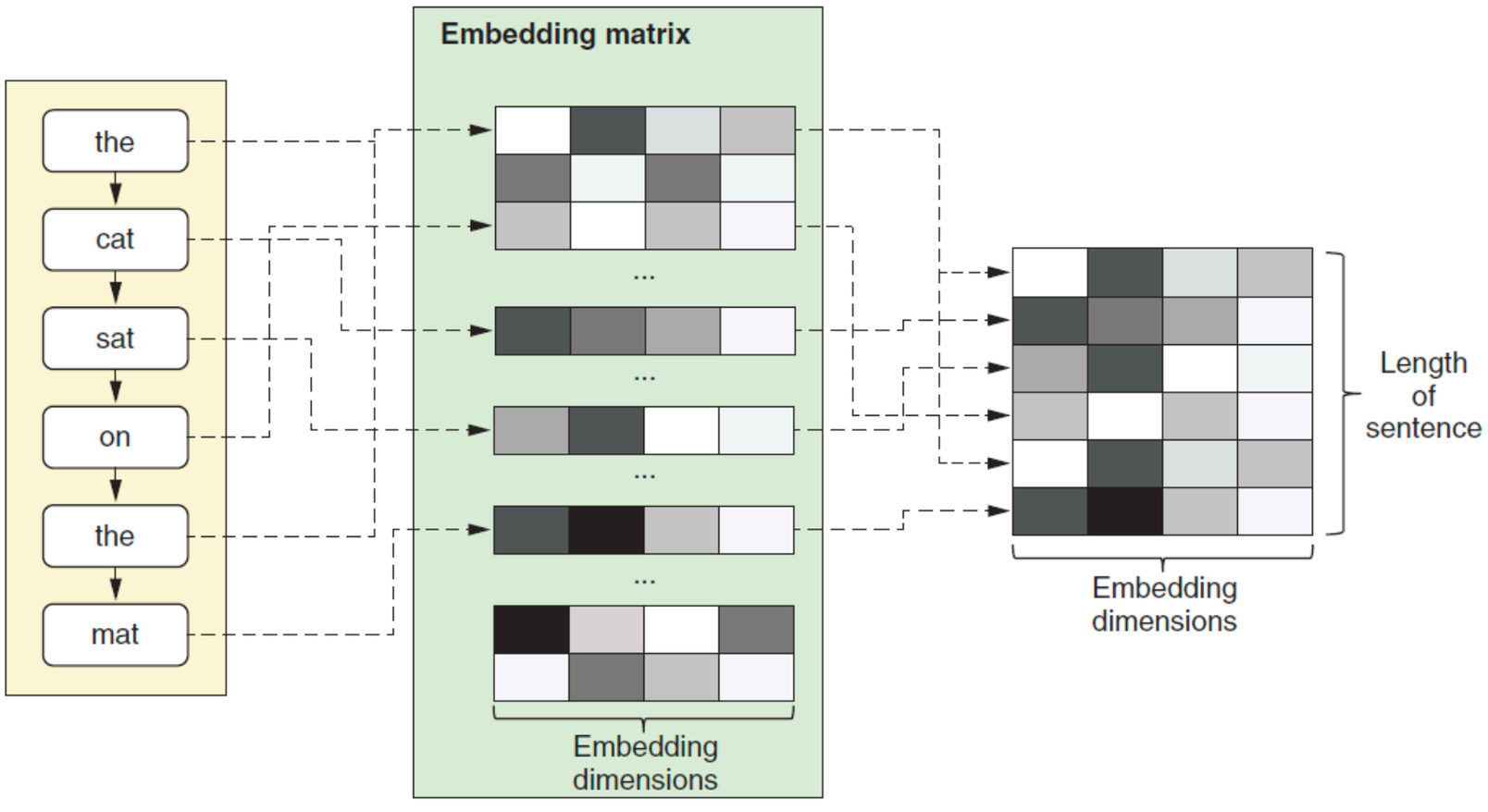
```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
embedding (Embedding)        (None, None, 64)          320000
_________________________________________________________________
dropout (Dropout)            (None, None, 64)          0
_________________________________________________________________
lstm (LSTM)                  (None, None, 64)          33024
_________________________________________________________________
lstm_1 (LSTM)                (None, 64)                33024
_________________________________________________________________
dense (Dense)                (None, 6)                 390
=================================================================
Total params: 386,438
Trainable params: 386,438
Non-trainable params: 0
```



Advancement: Embedding Layers

The model begins with an **embedding layer** which turns the input integer indices into the corresponding word vectors.

Word embedding is a way to represent a word as a vector. Word embeddings allow the value of the vector's element to be trained. After training, words with similar meanings often have the similar vectors.



https://djajafer.medium.com/multi-class-text-classification-with-keras-and-lstm-4c5525bef592